

Deep Learning — Assignment 12

Assignment for week 12 of the 2023 Deep Learning course (NWI-IMC070) of the Radboud University.

Names: Vojko Pjanovic (s1084416); Carmen Gascó Gálvez (s1098566)

Group: 10

Instructions:

- Fill in your names and the name of your group.
- Answer the questions and complete the code where necessary.
- Keep your answers brief, one or two sentences is usually enough.
- Re-run the whole notebook before you submit your work.
- Save the notebook as a PDF and submit that in Brightspace together with the `.ipynb` notebook file.
- The easiest way to make a PDF of your notebook is via File > Print Preview and then use your browser's print option to print to PDF.

Objectives

In this assignment you will

1. Implement GradCAM
2. Investigate the important features for certain inputs.
3. Investigate variants of GradCAM

Required software

As before you will need these libraries:

- `torch` and `torchvision` for PyTorch,
- `d2l`, the library that comes with [Dive into deep learning](#) book.

All libraries can be installed with `pip install`.

```
In [ ]: %config InlineBackend.figure_formats = ['png']
        %matplotlib inline

import os
from d2l import torch as d2l
import PIL
import urllib.request
import numpy as np
```

```

from matplotlib import colormaps
import matplotlib.pyplot as plt
import pandas as pd
import torch
from torch import nn
from torch.nn import functional as F
from torchvision import transforms
from torchvision.transforms.functional import to_pil_image
from torchvision.models import resnet50, ResNet50_Weights
from torchvision.io.image import read_image

device = 'cpu'

```

12.1 A pretrained network (4 points)

In this assignment we will be working with a pre-trained network. These are available in torchvision. Look at [the documentation](#) for more information on pretrained models.

In this assignment we will use the ResNet50 model.

(a) Load a pretrained ResNet50 model, with default weights

```

In [ ]: weights = ResNet50_Weights.DEFAULT
model = resnet50(weights=weights)
# model = model.to(device)
model.eval();

```

Next we will download a couple of random images, which we will use for the rest of this notebook.

(b) Run the code below to download the test images

```

In [ ]: # Download some images
urls = [
    'http://images.cocodataset.org/test-stuff2017/000000006149.jpg',
    'https://farm8.staticflickr.com/7020/6810252887_01e3d8e4e6_z.jpg',
    'http://images.cocodataset.org/val2017/000000039769.jpg',
    'http://images.cocodataset.org/train2017/000000140285.jpg',
    # Some more images to play around with
    'https://farm1.staticflickr.com/35/67223286_72fce12163_z.jpg',
    'https://farm4.staticflickr.com/3587/3508226585_268a2e5b9b_z.jpg',
    'https://farm1.staticflickr.com/6/6947166_ba80959bbb_z.jpg',
    'https://farm3.staticflickr.com/2325/5821134041_b96fef0946_z.jpg',
    'https://farm1.staticflickr.com/115/296513905_ddb2cf6438_z.jpg',
    'https://farm8.staticflickr.com/7020/6810252887_01e3d8e4e6_z.jpg',
]

def load_url(url):
    filename = os.path.basename(url)
    if not os.path.exists(filename):
        urllib.request.urlretrieve(url, filename)
    return read_image(filename)

images = [load_url(url) for url in urls]

```

The code below shows these images.

```
In [ ]: plt.figure(figsize=(15,15))
        for i, image in enumerate(images):
            plt.subplot(1,len(images), i+1)
            plt.imshow(transforms.ToPILImage()(image))
            plt.axis('off')
        plt.tight_layout()
```



The pretrained model was trained with a specific image preprocessing. So we should use the same:

```
In [ ]: preprocess = weights.transforms()
```

(c) Preprocess the images, and store them in a single tensor. (1 point)

Hint: `torch.stack` can be useful.

Note: you may get a warning about parameters of transform, you can ignore it.

```
In [ ]: x = torch.stack([preprocess(image) for image in images]).to(device)
```

/home/vojkinator/mambaforge/lib/python3.10/site-packages/torchvision/transforms/functional.py:1603: UserWarning: The default value of the antialias parameter of all the resizing transforms (Resize(), RandomResizedCrop(), etc.) will change from None to True in v0.17, in order to be consistent across the PIL and Tensor backends. To suppress this warning, directly pass antialias=True (recommended, future default), antialias=None (current default, which means False for Tensors and True for PIL), or antialias=False (only works on Tensors - PIL will still use antialiasing). This also applies if you are using the inference transforms from the model's weights: update the call to weights.transforms(antialias=True).
warnings.warn(

```
In [ ]: # Verify that the batch has the right shape
        assert(x.shape == torch.Size([len(images), 3, 224, 224]))
```

It is also useful to be able to undo the preprocessing, at least the normalization and the dimension permutation, so that we can plot the preprocessed images as well.

```
In [ ]: def unnormalize(x):
        """Make a preprocessed image showable."""
        x = torch.permute(x, [1,2,0]) # Make the color channels the last dimension
        x = x * torch.tensor(preprocess.std)[None,None,:]
        x = x + torch.tensor(preprocess.mean)[None,None,:]
        return x
```

(d) Plot the pre-processed images

```
In [ ]: plt.figure(figsize=(15,15))
        for i, image in enumerate(x):
```

```
plt.subplot(1,len(x), i+1)
plt.imshow(unnormailze(image.cpu()))
plt.axis('off')
plt.tight_layout()
```



(e) Compare the preprocessed images to the original. How does the default preprocessing make all the images the same size?

The preprocessing resizes the images to a default size based on the weights.transforms from the network

(f) Run the classifier on the batch of images.

(1 point)

```
In [ ]: y = model(x.to("cpu"))
        y.shape, y
```

```
Out[ ]: (torch.Size([4, 1000]),
        tensor([[ -0.0551,  0.1077,  0.2430, ...,  0.0189, -0.3340,  0.2478],
                [ -0.2221,  0.0183,  0.3214, ..., -0.6264,  0.0241,  0.7661],
                [ -0.1122, -0.6868,  0.2221, ..., -0.0857, -1.0869, -0.5299],
                [ -0.1739,  0.2037, -0.2916, ..., -0.1072, -0.4872, -0.0530]],
        grad_fn=<AddmmBackward0>))
```

The classifier predicts a logit scores, represing the likelihood of 1000 classes.

You can use the `torch.topk` function to get a list of the predicted labels with the highest score. This function returns a tuple with two elements, `result.indices` is the indices, and `result.values` contains the correspondig scores.

By themselves, these indices don't tell you what the predicted class actually is. For that you can use the following function:

```
In [ ]: def category(index):
        """Return the textual category for some predictions"""
        return np.array(weights.meta["categories"])[index]
```

(g) Create a table with the textual label for the top 5 predicted labels for each image.

(1 point)

Hint: `pd.DataFrame` is an easy way of producing a nice looking table.

```
In [ ]: all_labels = []
        for i in range(len(y)):
            labels = torch.topk(y[i],k=5)[1]
            labels = labels.detach().numpy()
            all_labels.append(category(labels))
```



```
In [ ]: df = pd.DataFrame(all_labels)
df.columns = ["Top 1", "Top 2", "Top 3", "Top 4", "Top 5"]
df.index = ['image 1', 'image 2', 'image 3', 'image 4']
df
```

```
Out[ ]:
```

	Top 1	Top 2	Top 3	Top 4	Top 5
image 1	racket	tennis ball	ping-pong ball	dugong	volleyball
image 2	American Staffordshire terrier	boxer	Staffordshire bullterrier	bull mastiff	tennis ball
image 3	tabby	tiger cat	Egyptian cat	remote control	sleeping bag
image 4	African elephant	tusker	Indian elephant	warthog	broccoli

(h) Are there any strange labels in the top 5 predictions for any of the images?
(1 point)

Yes.

The tennis player's 4th is dugong (a weird animal)

the cats have remote control and sleeping bag

the elephant has broccoli

12.2 Hooking into the network (4 points)

To understand and visualize which part of the images are 'causing' the predicted labels, we can use [GradCAM](#).

GradCAM needs to know the activations and output gradients of the last layer in the model, just before the global pooling layer. The activations are computed during forward propagation (`model.forward()`), and the gradients are computed when calling `y.backward()` , but neither of them are saved anywhere.

To capture the activations and gradients we need to use hooks, which allow us to register a function that gets called every time a module's `forward` or `backward` function is called.

Take a look at `torch.Module.register_forward_hook` and `torch.Module.register_full_backward_hook` .

(a) Complete the code below. (2 points)

You can store the activations and gradients as a member variable of the layer, or you can use a global variable.

```
In [ ]: def add_hooks(layer):
        """
        Add hooks to a layer that store the activations (the output of the layer) in
        as well as the gradient wrt. the output in the backward pass
        """

        def forward_hook(layer, args, output):
            # This function will be called after the forward pass.
            # args are the inputs of the layer
            # output is the output of the layer

            layer.activations = output

        def backward_hook(layer, grad_input, grad_output):
            # This function is called after the backward pass
            # grad_input is a *tuple* that contains the gradients wrt the inputs.
            # grad_output is a *tuple* the gradient the gradients wrt the outputs.
            layer.gradients = grad_output[0]

        # Remove old hooks (if any)
        remove_hooks(layer)

        # Register new hooks
        layer.forward_hook = layer.register_forward_hook(forward_hook)
        layer.backward_hook = layer.register_full_backward_hook(backward_hook)

    def remove_hooks(layer):
        if hasattr(layer, 'forward_hook'):
            layer.forward_hook.remove()
        if hasattr(layer, 'backward_hook'):
            layer.backward_hook.remove()
```

(b) Which layer of the ResNet50 model should the hooks be applied to? (1 point)

Hint: you can `print` a model, and you can access a named layer using `model.layer`

```
In [ ]: hooked_layer = model.layer4[-1].conv3
```

(c) Add the hooks to the layer. (no points)

```
In [ ]: add_hooks(hooked_layer)
```

(d) Verify that the hooks work, by completing and running the code below.(1 point)

```
In [ ]: y = model(x[[0]]) # x[[0]] selects the first image in the batch

        # Get the top predicted class
        top_class = torch.argmax(y, dim=1)

        # Run backward for the logit of the top predicted class
        model.zero_grad() # Zero out any existing gradients
        y[0, top_class].backward() # Backward pass for the top class logit

        # Assuming 'target_layer' is the layer to which hooks were added
        layer_activations = hooked_layer.activations
        layer_gradients = hooked_layer.gradients

        assert layer_activations.shape[1:] == torch.Size([2048, 7, 7]), "Activations has
```

```

assert layer_gradients.shape == layer_activations.shape, "Gradients have the wrong shape"
assert torch.mean(layer_activations) != 0, "Activations should not be zero"
assert torch.mean(layer_gradients) != 0, "Gradients should not be zero"
assert torch.mean(layer_activations - layer_gradients) != 0, "Gradients should not be zero"
assert top_class == 752, "Did you use the right image"

```

12.3 GradCAM (9 points)

Now we are ready to implement GradCAM.

(a) GradCAM uses 'neuron importance weights', based on the gradients. Compute these importance weights. (1 point)

See equation (1) of [the paper](#).

```

In [ ]: importance_weights = torch.mean(layer_gradients, dim=(2,3), keepdim=False).squeeze_(-1)
assert importance_weights.shape == torch.Size([2048])

```

(b) Combine the importance weights with the activations to get the gradcam map. (2 point)

See equation (2) of [the paper](#).

Hint: To combine two tensors they need to have the same shape. You can make them line up by adding new dimensions to a tensor using `tensor[:,None,:,None]`, where `None` is a new dimension, while `:` is an existing dimension.

```

In [ ]: importance_weights.shape, layer_activations.shape

Out[ ]: (torch.Size([2048]), torch.Size([1, 2048, 7, 7]))

In [ ]: weighted_activations = importance_weights[:, None, None] * layer_activations[0,:]
gradcam_map = weighted_activations.sum(dim=0)
gradcam_map = torch.nn.functional.relu(gradcam_map)

assert gradcam_map.shape == torch.Size([7,7])
assert torch.min(gradcam_map) >= 0

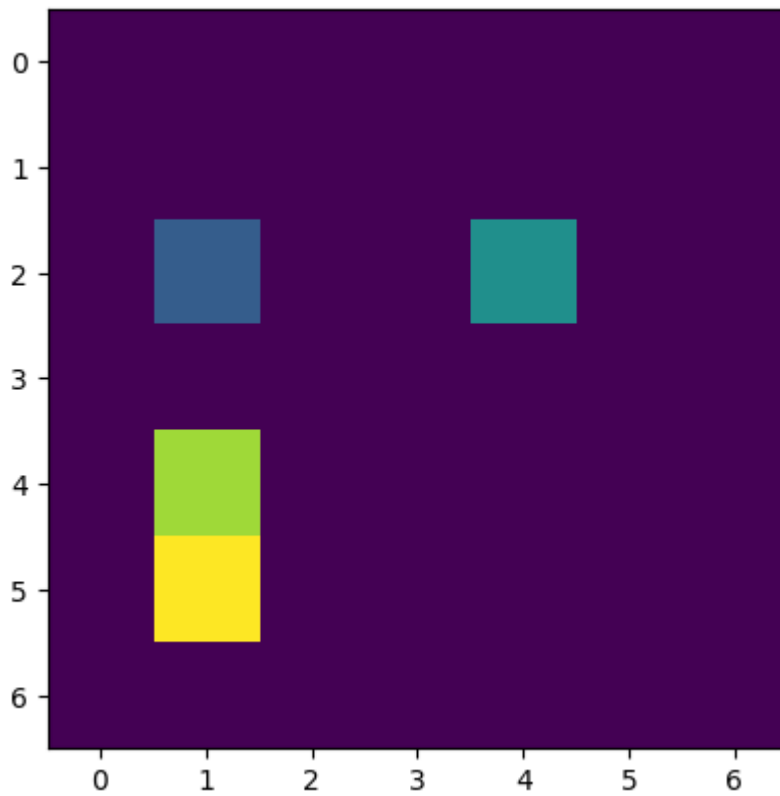
```

(c) Plot the gradcam map.

```

In [ ]: plt.imshow(gradcam_map.detach().cpu());

```

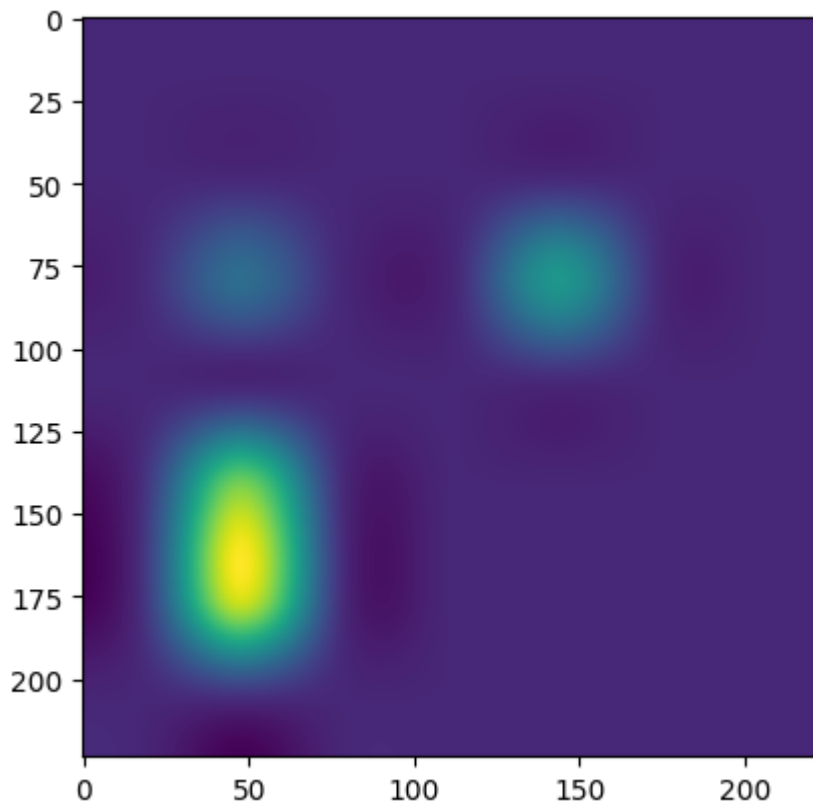


We can resize the map using interpolation to make it smoother, and to align it with the input image.

(d) Plot the resized and smoothed GradCAM map.

```
In [ ]: def resize_map(map, size=(224,224)):
          return np.asarray(to_pil_image(map.detach().cpu(), 'F').resize(size))

plt.imshow(resize_map(gradcam_map));
```



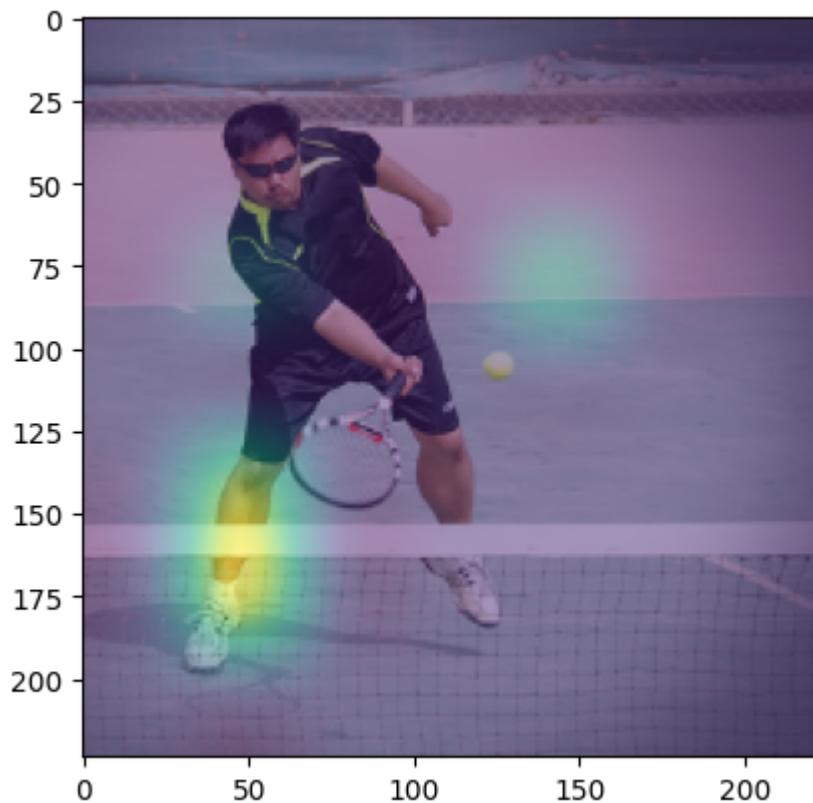
(e) Overlay the gradcam map over the input image.

(1 point)

You can overlay two images by drawing the second one semi-transparent with `alpha=0.5`.

Expected output: the highlighted region should correspond to the predicted label. Although the alignment might be slightly off.

```
In [ ]: plt.imshow(unnormalize(x[0].cpu()));  
plt.imshow(resize_map(gradcam_map), alpha=0.5);
```



We needed several steps to compute a gradCAM visualisation. Let's clean up the code by encapsulating it in a function.

(f) Complete the code below.

(1 point)

Hint: the model always works on a batch of images, you can turn a single image into a batch by adding a new dimension, `image[None]`.

```
In [ ]: def gradcam(image, index):
    """
    Compute a GradCAM map for the given input image, and class index.
    """
    image = image[None]
    y = model(image)
    top_class = torch.argmax(y, dim=1)
    model.zero_grad()
    y[0, index].backward()

    layer_activations = hooked_layer.activations
    layer_gradients = hooked_layer.gradients

    importance_weights = torch.mean(layer_gradients, dim=(2,3), keepdim=False).s

    weighted_activations = importance_weights[:, None, None] * layer_activations

    gradcam_map = weighted_activations.sum(dim=0)
    gradcam_map = torch.nn.functional.relu(gradcam_map)

    return gradcam_map

assert torch.any(gradcam(x[0],0) != gradcam(x[1],0)), "Are you using the incorre
assert torch.all(gradcam(x[0],0) == gradcam(x[0],0)), "Calling the function twic
```


(g) Complete the code below to create a class activation map for the top 5 label for each of the images. (2 point)

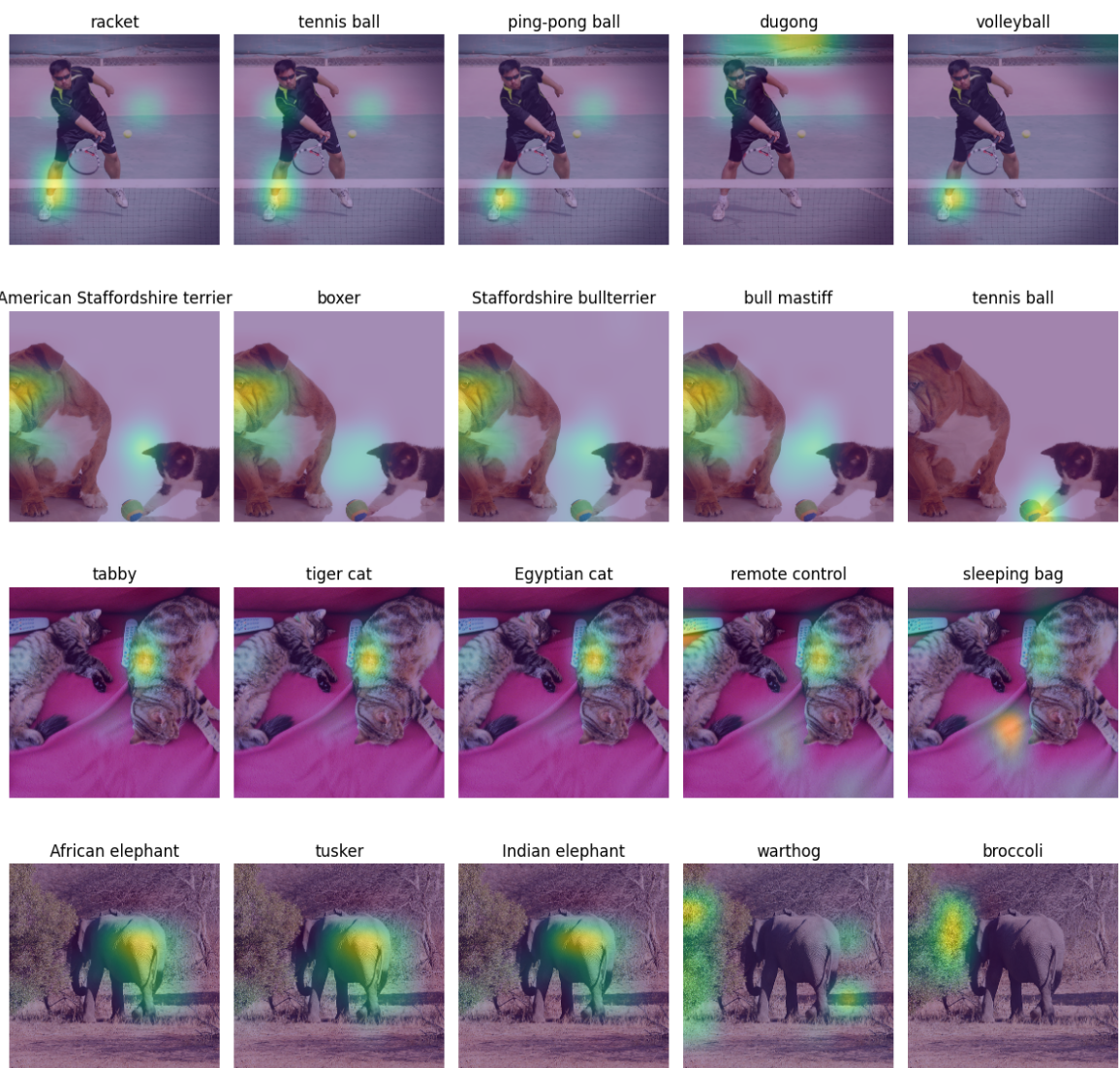
```
In [ ]: plt.figure(figsize=(12,12))
for i, image in enumerate(x):
    top5_indices = torch.topk(model(image[None]),k=5)[1].detach().numpy()

    for j, index in enumerate(top5_indices[0]):
        plt.subplot(len(images),5, i*5 + j + 1)

        gradcam_map = gradcam(image, index)

        plt.imshow(unnormalize(image.cpu()))
        plt.imshow(resize_map(gradcam_map), alpha=0.5);

        plt.title(category(index))
        plt.axis('off')
plt.tight_layout()
```



(h) Do the gradcam maps make sense? Do the highlighted areas roughly correspond to the location of objects in the image? Mention specific examples. (1 point)

To some extent, yes.

For the tennis player, it does not seem to work very well; the map seems to be misplaced repeatedly at the feet of the player. However, the objects are still correctly found.

For the second image, I initially said that the tennis ball didn't make sense as I overlooked it, but now it pointed it out explicitly -- good performance.

For the last image, broccoli now also makes sense. The gradcam map explicitly shows the tree only, which is rather appropriately categorized as broccoli.

(i) Do the highlighted areas correspond *exactly* to the location of objects in the image? Why / why not? (1 point)

In most cases not *exactly*. This of course makes sense given that it's an approximation, a neural network. However, for the tennis ball and the broccoli for the second and third image respectively it is very localized, which makes sense as well given that it's a CNN, with specific receptive fields at different scales.

In the previous image with the tennis player, we see that there are appropriate sized blobs, but they are misplaced. It might also be due to the fact that we are resizing images and the gradcam output that the blobs do not exactly match the object they are representing

12.4 Variants (7 points)

GradCAM includes a non-linear activation function. The result is that the GradCAM map that is produced only contains positive values. But perhaps the negative values also contain useful information?

(a) Copy and modify the code from part 12.3f and g, by negating the input to the non-linearity. (1 point)

```
In [ ]: def gradcam_negative(image, index):
        """
        Compute a GradCAM map for the given input image, and class index.
        """
        image = image[None]
        y = model(image)
        top_class = torch.argmax(y, dim=1)
        model.zero_grad()
        y[0, index].backward()

        layer_activations = hooked_layer.activations
        layer_gradients = hooked_layer.gradients

        importance_weights = torch.mean(layer_gradients, dim=(2,3), keepdim=False).s

        weighted_activations = importance_weights[:, None, None] * layer_activations

        gradcam_map = - weighted_activations.sum(dim=0)
        gradcam_map = torch.nn.functional.relu(gradcam_map)
```

```

return gradcam_map

plt.figure(figsize=(12,12))
for i, image in enumerate(x):
    top5_indices = torch.topk(model(image[None]),k=5)[1].detach().numpy()

    for j, index in enumerate(top5_indices[0]):
        plt.subplot(len(images),5, i*5 + j + 1)

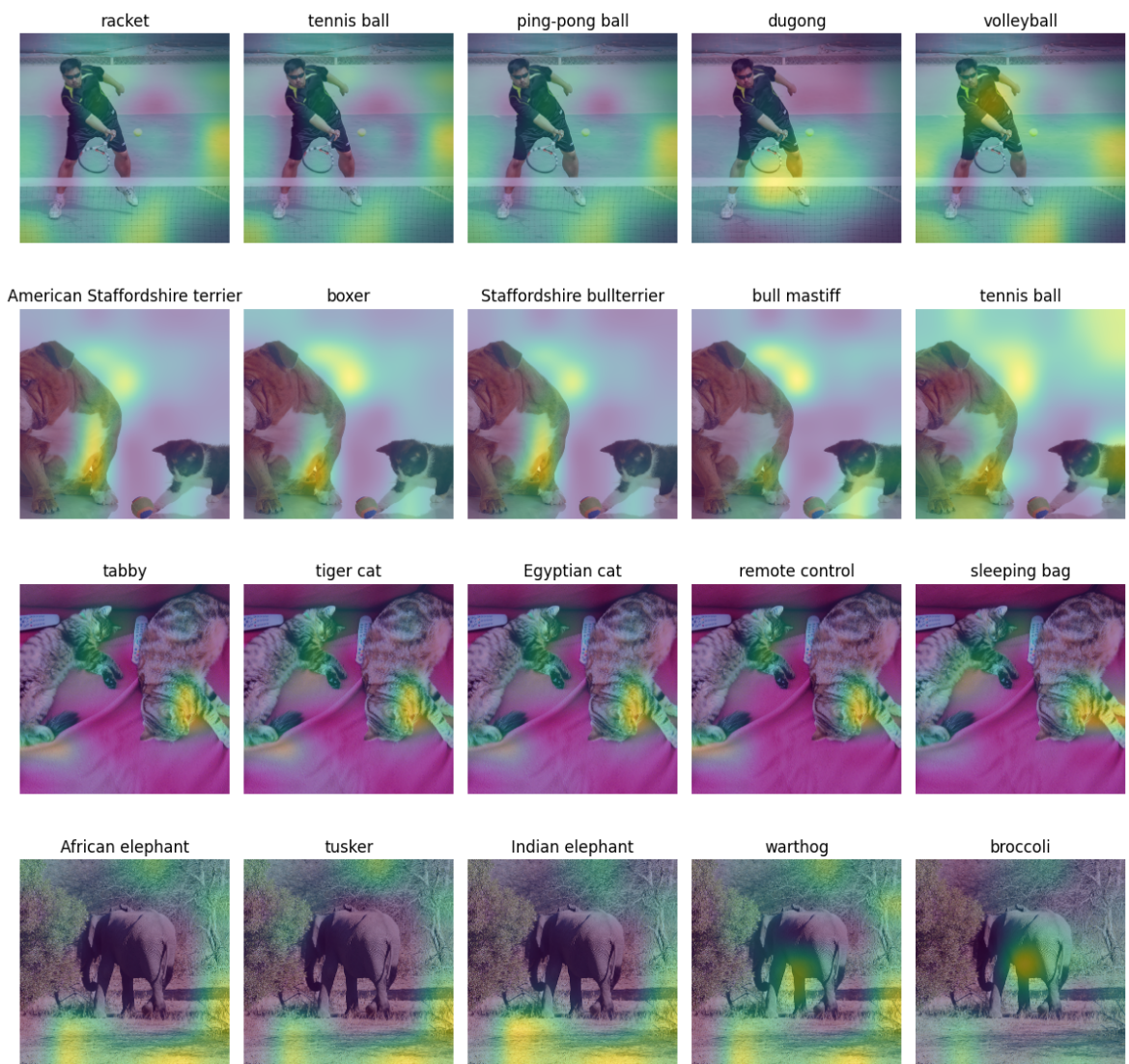
        gradcam_map = gradcam_negative(image, index)

        plt.imshow(unnormailze(image.cpu()))
        plt.imshow(resize_map(gradcam_map), alpha=0.5);

        plt.title(category(index))
        plt.axis('off')
plt.tight_layout()

assert torch.all((gradcam(x[0],123) > 0) != (gradcam_negative(x[0],123) > 0)), "

```



(b) What are the areas that are highlighted now?

(1 point)

In image 2 and 4 it is mostly around the objects of interest, whereas for image 3 it does explicitly highlight the cats better than the normal gradcam! For the elephant the last 2

renditions seem to highlight the elephant nicely, and for the tennis player in the 4th and 5th rendition relevant features seem to be highlighted.

(c) Plot the negative gradcam map for the bottom 5 predictions instead of the top 5. (1 point)

Hint: the bottom 5 of `x` is the top 5 of `-x`.

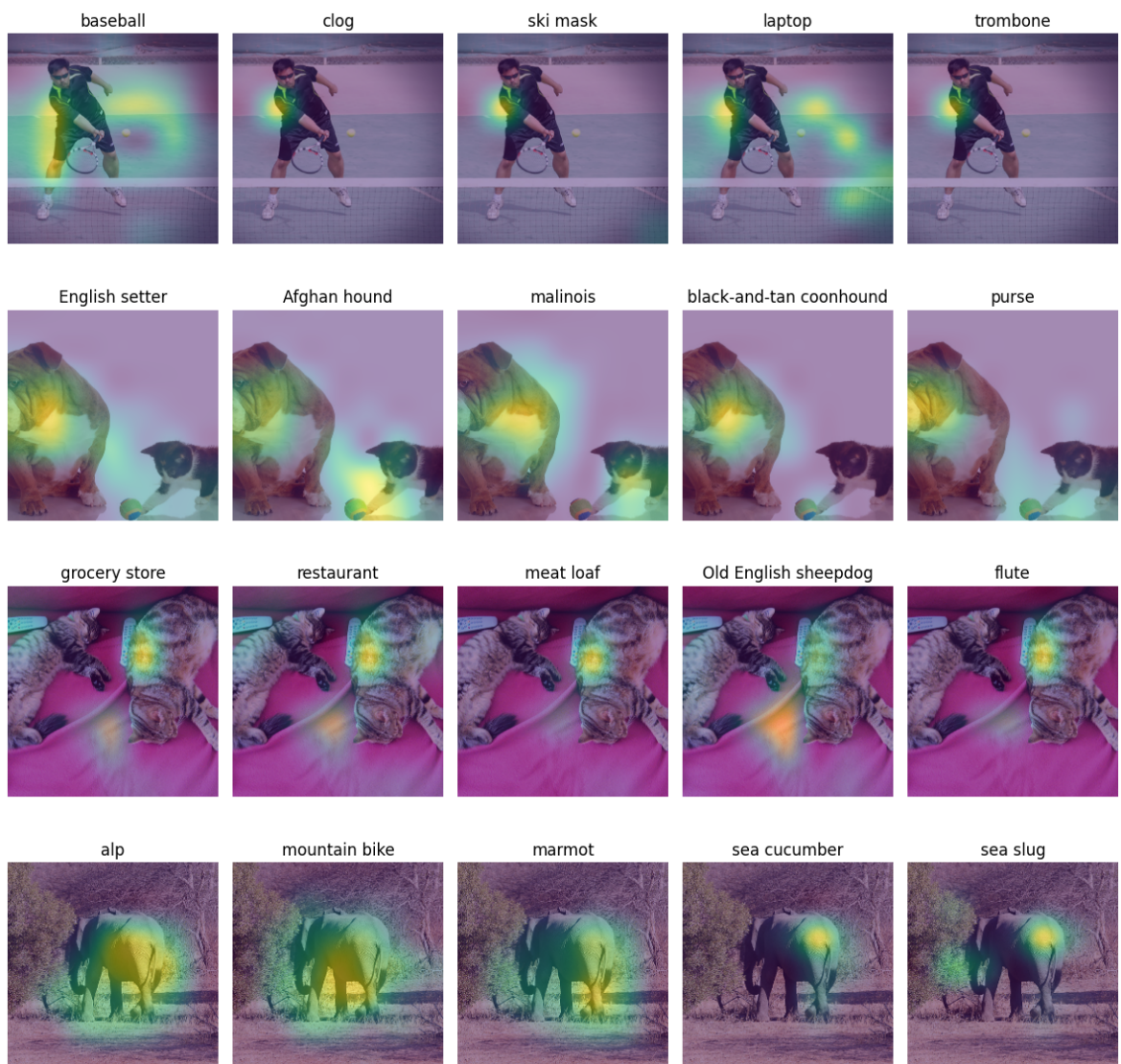
```
In [ ]: plt.figure(figsize=(12,12))
for i, image in enumerate(x):
    top5_indices = torch.topk(-model(image[None]),k=5)[1].detach().numpy()

    for j, index in enumerate(top5_indices[0]):
        plt.subplot(len(images),5, i*5 + j + 1)

        gradcam_map = gradcam_negative(image, index)

        plt.imshow(unnormalize(image.cpu()))
        plt.imshow(resize_map(gradcam_map), alpha=0.5);

        plt.title(category(index))
        plt.axis('off')
plt.tight_layout()
```



(d) What can you learn from these maps? Give at least 1 concrete example. (1 point)

This second set of maps seems to highlight what was left out in the previous set of maps, and they coincide rather nicely with the top indices of normal gradcam. For example, the buttocks of the elephant were highlighted in the normal gradcam as well, which is the case now as well.

Instead of looking at the last layer before global pooling, we could in theory also apply GradCAM at a different layer of the network. Perhaps that can give a higher resolution map?

(e) Adapt the gradcam code to take the model and layer of the model as parameters (1 point)

Note: you may need to add hooks at this point.

```
In [ ]: def gradcam_at(model, layer, image, index):
        """
        Compute a GradCAM map at the given layer of the given model
        """

        if layer == 1:
            hooked_layer = model.layer1[-1].conv3
        elif layer == 2:
            hooked_layer = model.layer2[-1].conv3
        elif layer == 3:
            hooked_layer = model.layer3[-1].conv3
        elif layer == 4:
            hooked_layer = model.layer4[-1].conv3

        add_hooks(hooked_layer)
        image = image[None]
        y = model(image)
        top_class = torch.argmax(y, dim=1)
        model.zero_grad()
        y[0, index].backward()

        layer_activations = hooked_layer.activations
        layer_gradients = hooked_layer.gradients

        importance_weights = torch.mean(layer_gradients, dim=(2,3), keepdim=False).s

        weighted_activations = importance_weights[:, None, None] * layer_activations

        gradcam_map = weighted_activations.sum(dim=0)
        gradcam_map = torch.nn.functional.relu(gradcam_map)

        remove_hooks(hooked_layer)

        return gradcam_map
```

(f) Now use gradcam at an intermediate layer of the model (1 point)

```
In [ ]: weights = ResNet50_Weights.DEFAULT
        model = resnet50(weights=weights)
        model.eval();
```

```
gradcam_map = gradcam_at(model, 3, x[0], 123)

assert gradcam_map.shape != torch.Size([7,7]), "Use a different layer"
```

```
In [ ]: plt.figure(figsize=(12,12))
for i, image in enumerate(x):
    top5_indices = torch.topk(model(image[None]),k=5)[1].detach().numpy()

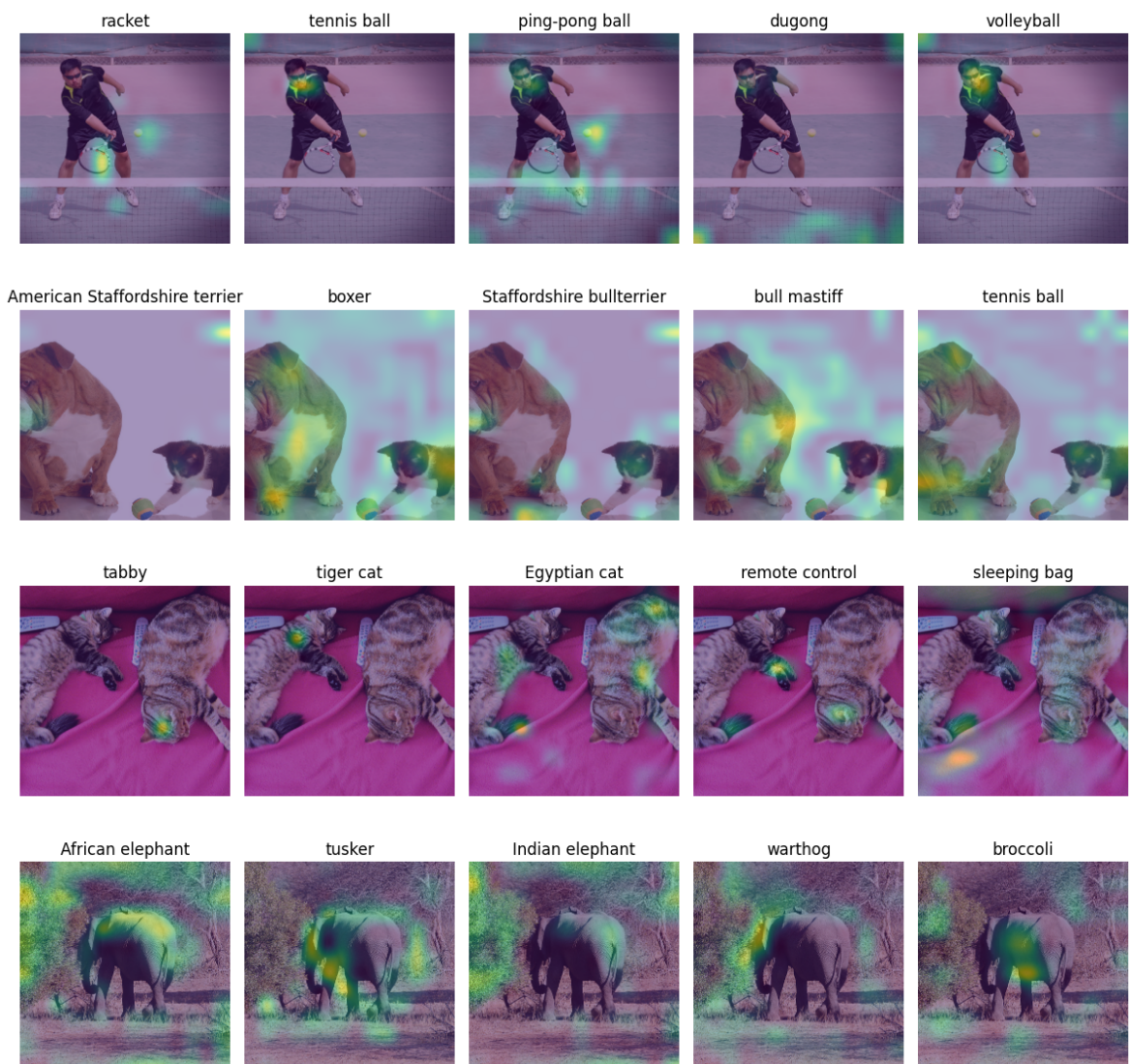
    for j, index in enumerate(top5_indices[0]):
        plt.subplot(len(images),5, i*5 + j + 1)

        # weights = ResNet50_Weights.DEFAULT
        # model = resnet50(weights=weights)
        # model.eval();

        gradcam_map = gradcam_at(model, 3, image, index)

        plt.imshow(unnormailze(image.cpu()))
        plt.imshow(resize_map(gradcam_map), alpha=0.5);

        plt.title(category(index))
        plt.axis('off')
plt.tight_layout()
```



(g) Do the gradcam maps for this earlier layer give a useful visualization? (1 point)

The maps do give useful visualizations; for the tennis player, the racket and pingpong ball (i.e. tennis ball) are nicely located. This is the strongest example which is evident in the output of the model, but missing when applying gradcam to the final layer.

Also the elephant has better some additional visualizations involved.

12.5 Of cats and dogs (5 points)

For the second image (`images[1]` , the one with the white background), there is an object that is clearly in the image, but it does not appear in the top 5 classes.

(a) Why is this object not detected in the top 5? (1 point)

The cat is not detected in the top 5, because the top 4 are taken only different classifications of the dog. The dog is much bigger than the cat, and the tennis ball is rather small. The cat is somewhere in between, and it is shifted somewhat and not very clearly directed towards the camera.

(b) How could you reduce the bias of the model for the second image, to make it also detect the other object? (1 point)

A bias can be added to manage the varying receptive fields. The dog is much bigger in the image, which might make the cat unrealistically small - it is at the feature level of the dog, and it is not in the shot. There should be a 'normalization' for the RF levels.

(c) Curuously, for this second image, the bottom 5 labels for this image also include dogs. Why would that happen? (1 point)

Because the model is discerning between different dogs for the dog in the image. The bottom 4 labels are the least likely type of dog.

(d) Create a GradCAM map for the missing object in the image. (1 point)

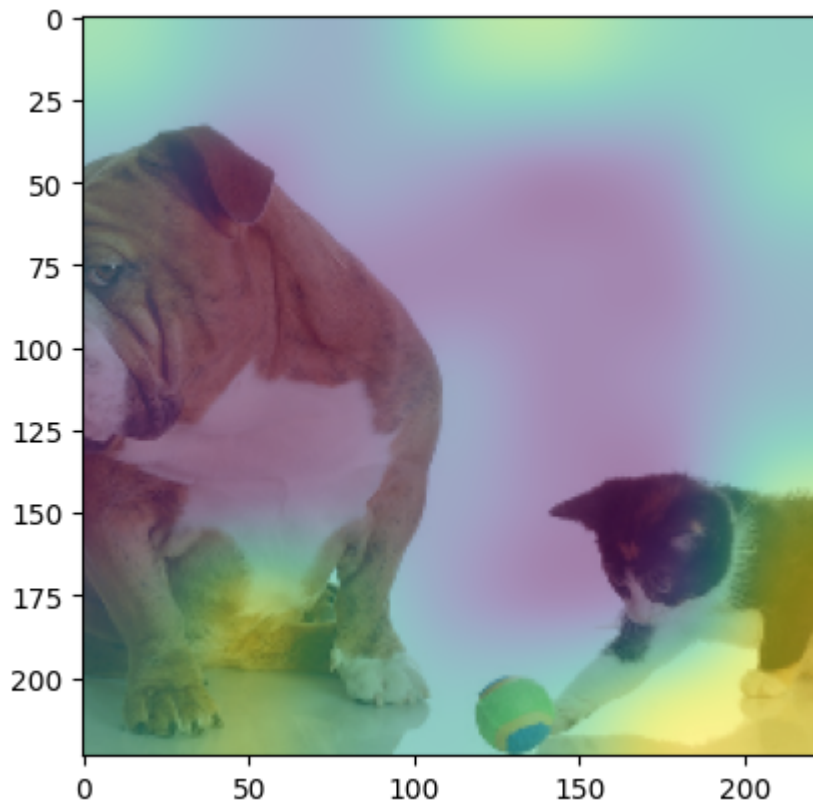
Hint: The third images contains some labels that you can use. You can also use the following function to get the corresponding class index.

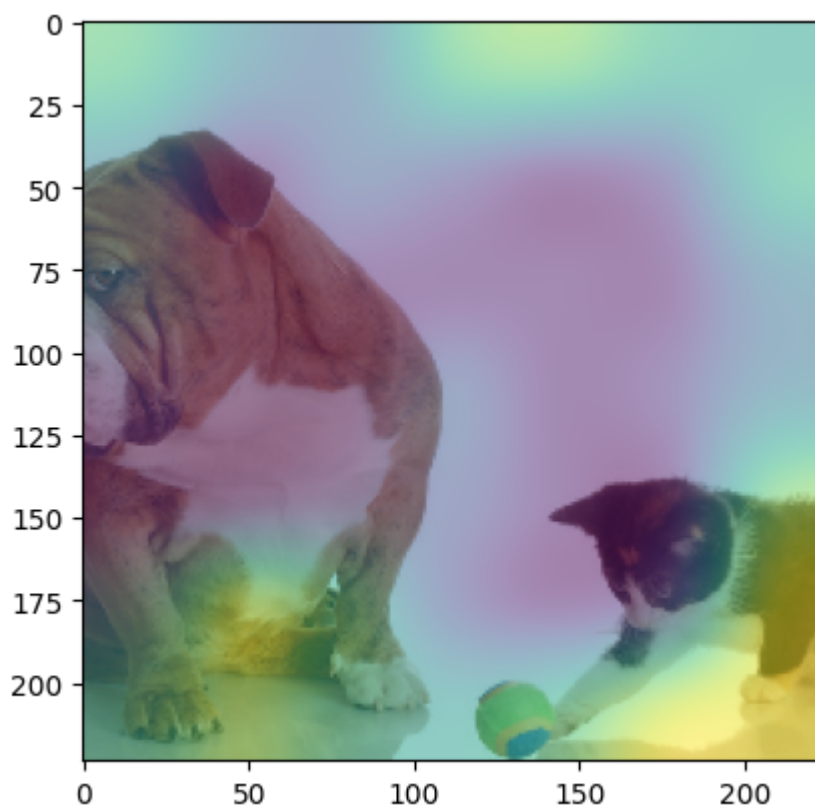
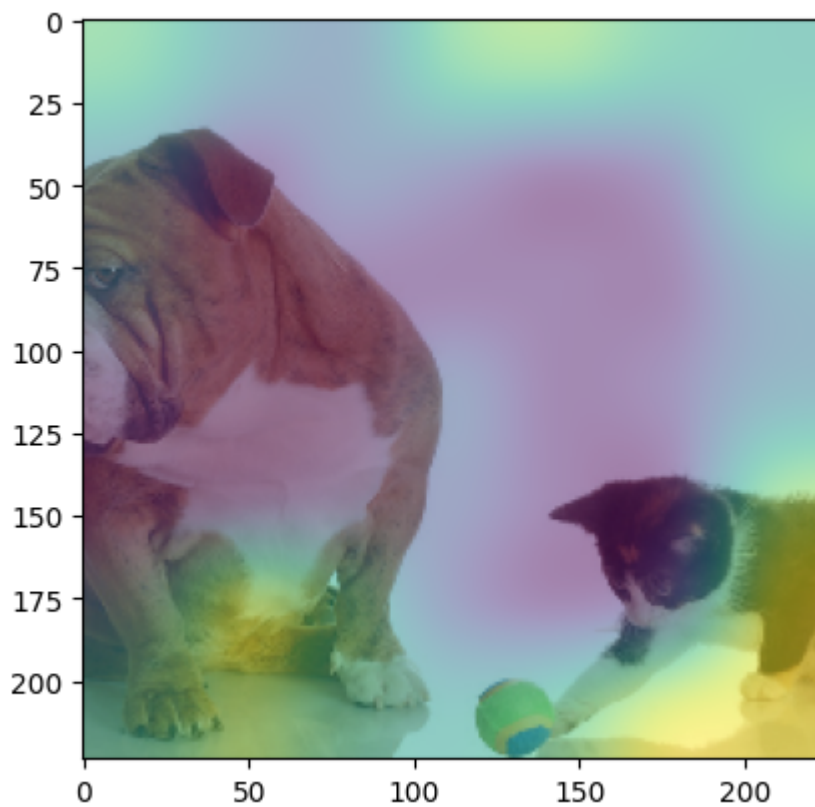
```
In [ ]: def find_category(query):
        """Find a category that has or contains the given name."""
        found = [i for (i,k) in enumerate(weights.meta["categories"]) if k.find(query)
        if len(found) == 1:
            return found[0]
        elif len(found) > 1:
            raise Exception("Multiple categories found: " + str(category(found)))
        else:
            raise Exception("No category found that matches: " + str(query))
```

```
In [ ]: plt.imshow(unnormalize(x[1]))
        gradcam_map = gradcam(x[1], find_category("tiger cat"))
```



```
plt.imshow(resize_map(gradcam_map),alpha=0.5);  
plt.show()  
  
plt.imshow(unnormalize(x[1]))  
gradcam_map = gradcam(x[1], find_category("tabby"))  
plt.imshow(resize_map(gradcam_map),alpha=0.5);  
plt.show()  
  
plt.imshow(unnormalize(x[1]))  
gradcam_map = gradcam(x[1], find_category("Siamese cat"))  
plt.imshow(resize_map(gradcam_map),alpha=0.5);  
plt.show()
```

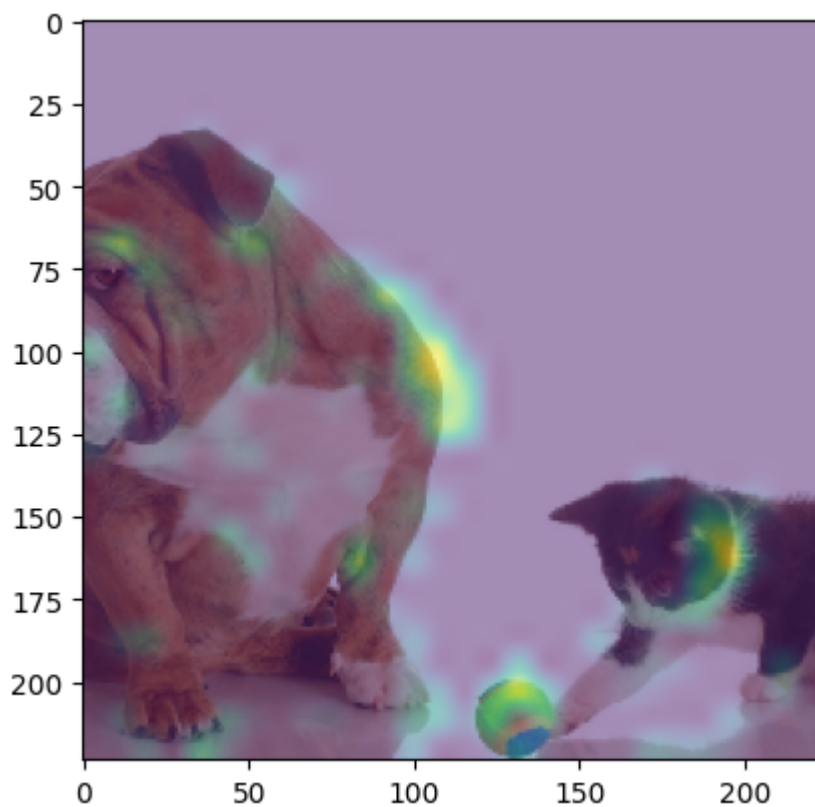
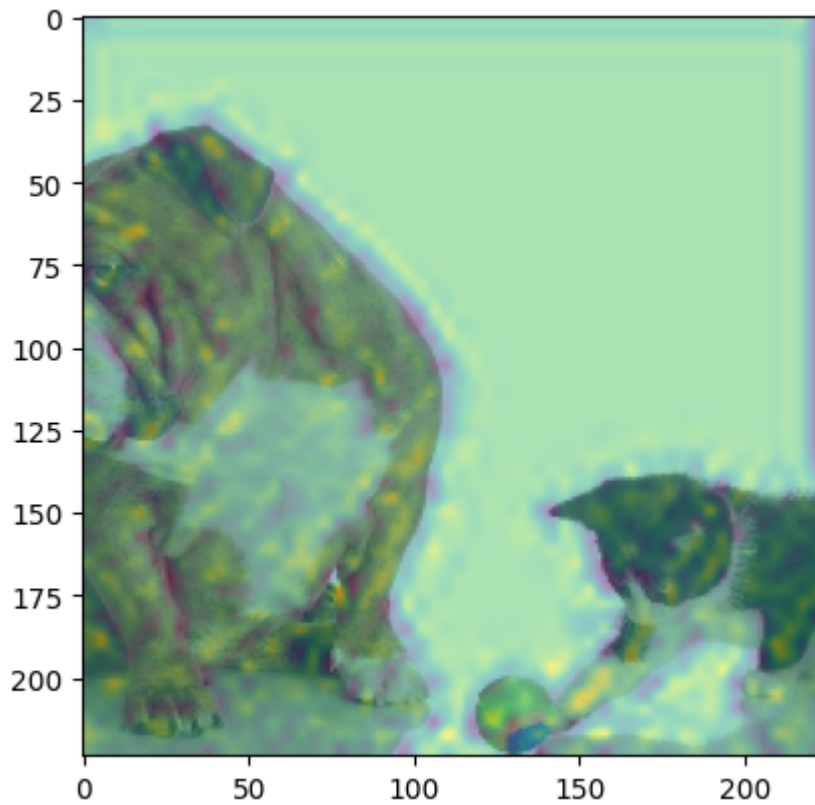


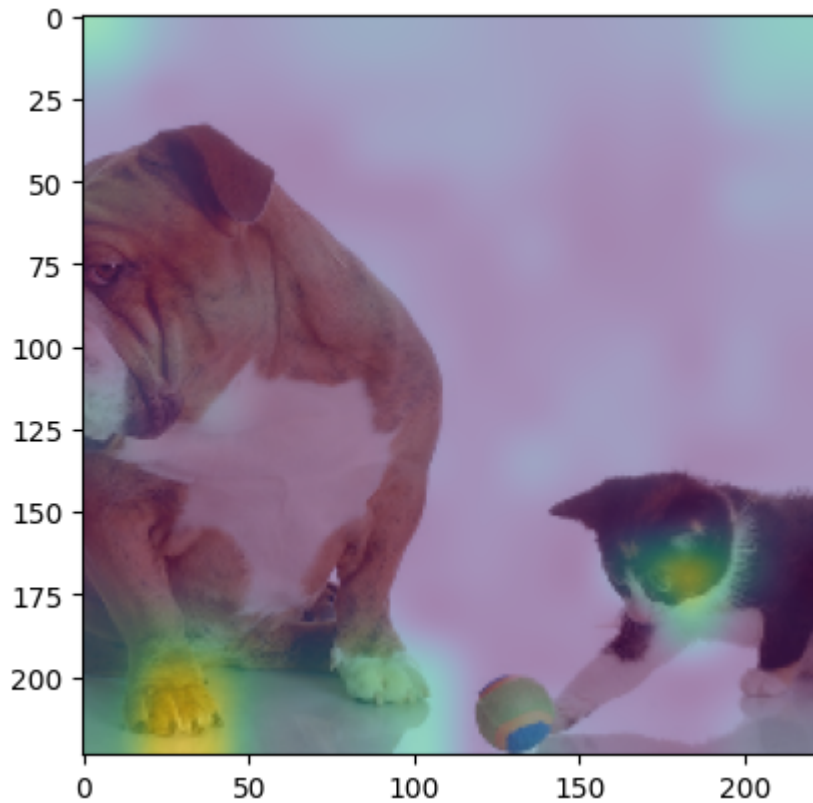


```
In [ ]: plt.imshow(unnormalize(x[1]))
gradcam_map = gradcam_at(model, 1, x[1], find_category("Siamese cat"))
plt.imshow(resize_map(gradcam_map),alpha=0.5);
plt.show()

plt.imshow(unnormalize(x[1]))
gradcam_map = gradcam_at(model, 2, x[1],find_category("Siamese cat"))
plt.imshow(resize_map(gradcam_map),alpha=0.5);
plt.show()
```

```
plt.imshow(unnormalize(x[1]))  
gradcam_map = gradcam_at(model, 3, x[1], find_category("Siamese cat"))  
plt.imshow(resize_map(gradcam_map), alpha=0.5);  
plt.show()
```





(e) Are you able to find this object when you go looking for it? Explain your answer briefly. (1 point)

To some extent, yes. It is closest to the cat, and not close to the dog or tennis ball. However, it does not nicely encompass the cat as would be wanted.

When using only the Siamese cat and then looking at the different layers, we see that in the third layer the cat's face is highlighted nicely, but the dog's paw is as well.

All in all, sort of.

The end

Well done! Please double check the instructions at the top before you submit your results.

This assignment has 29 points.

Version 3770c63 / 2023-12-04